



Servicio de notificaciones

Ejemplo de documentación técnica



AUTOR

Equipo de Documentación
docs@example.com

FECHA

15/01/2026

VERSIÓN

1.0

ORGANIZACIÓN

Acme Corp

Índice

Resumen	3
Arquitectura	3
Componentes	4
Flujo de un mensaje	4
Decisiones de diseño.....	4
Requisitos de despliegue	4
Requisitos previos	5
Objetivos de calidad	5
Documentos relacionados.....	5
Glosario	6

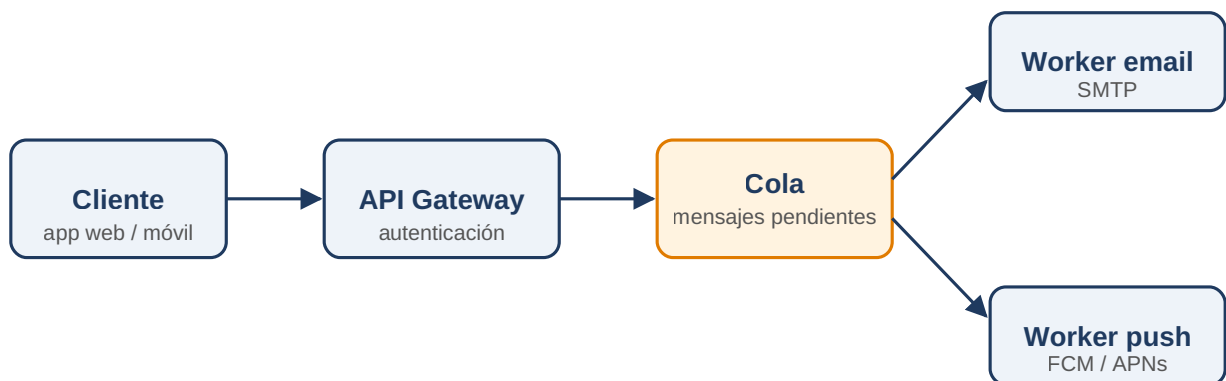
Resumen

El **servicio de notificaciones** recibe eventos de otras aplicaciones y los entrega al usuario por el canal que haya elegido (correo o notificación push). Este documento describe su arquitectura, sus componentes y las decisiones de diseño más importantes.

Este PDF se ha generado a partir de una nota de Obsidian con `obsidian2pdf`. Sirve de ejemplo de **documentación técnica**: índice automático, tablas, imágenes, enlaces entre documentos y contenido reutilizado.

Arquitectura

Los clientes publican mensajes a través de una pasarela. La pasarela valida la petición y la deja en una cola; después, un *worker* por canal la recoge y la entrega.



Componentes

Componente	Responsabilidad	Tecnología
API Gateway	Autenticación, validación y límites de uso	Nginx + Lua
Cola	Almacena los mensajes pendientes con reintentos	Redis Streams
Worker email	Renderiza la plantilla y envía por SMTP	Python
Worker push	Entrega a FCM y APNs	Go
Base de datos	Preferencias de usuario e histórico	PostgreSQL 16

Flujo de un mensaje

1. El cliente envía `POST /v1/messages` con el destinatario y la plantilla.
2. La pasarela comprueba el token y responde `202 Accepted`.
3. El mensaje entra en la cola con un identificador único.
4. El *worker* del canal lo procesa y registra el resultado.
5. Si falla, se reintenta hasta 5 veces con espera exponencial.

Decisiones de diseño

- **Entrega al menos una vez.** Es preferible duplicar un aviso a perderlo; los clientes deben tolerar duplicados usando el identificador del mensaje.
- **Un worker por canal.** Un fallo en el proveedor de correo no afecta a las notificaciones push.
- **Plantillas versionadas.** Cada mensaje guarda la versión de la plantilla con la que se generó.

Requisitos de despliegue

Estos son los requisitos para desplegar la parte de integración continua. Se reutilizan directamente desde otra nota con `![[nota#sección]]`, por lo que si cambian allí, cambian aquí:

Requisitos previos

Requisito	Versión mínima	Comprobación
Docker Engine	24.0	<code>docker --version</code>
Docker Compose	2.20	<code>docker compose version</code>
Memoria libre	2 GB	<code>free -h</code>
Puertos libres	8080 y 50000	<code>ss -tlnp</code>

Además necesitas permisos para ejecutar comandos de Docker (pertenecer al grupo `docker`).

Objetivos de calidad

Métrica	Objetivo	Estado
Disponibilidad mensual	99,9 %	Cumplido
Latencia de encolado (p95)	< 50 ms	Cumplido
Mensajes entregados en 1 min	99 %	En curso

Pendiente de completar:

- ☒ Definir el contrato de la API
- ☒ Diagrama de arquitectura
- ☐ Prueba de carga con 10 000 mensajes por minuto
- ☐ Documentar la política de retención del histórico

Documentos relacionados

- [Documentación de la API REST](#): cómo consumir el servicio desde código, con ejemplos en varios lenguajes.
- [Despliegue de Jenkins en Docker](#): guía paso a paso con avisos y advertencias.

Estos enlaces apuntan al PDF de cada nota **con una ruta relativa**, así que siguen funcionando si mueves la carpeta completa.

Glosario

Idempotente

Una operación que produce el mismo resultado aunque se ejecute varias veces.

Worker

Proceso que consume mensajes de una cola y los procesa en segundo plano.

Backoff exponencial

Estrategia de reintentos en la que el tiempo de espera se duplica tras cada fallo.